BÁO CÁO ĐỒ ÁN · IAD591.22 · PHÁT TRIỂN ỨNG DỤNG IOT

Smart Room Digital Twin

Một căn phòng tự cảm nhận nhiệt độ, độ ẩm, ánh sáng, chất lượng không khí và sự hiện diện của người. Phòng tự quyết định trạng thái an toàn ngay trên vi điều khiển và có một bản sao số chạy live trên web và điện thoại, điều khiển được bằng nút bấm hoặc giọng nói.

Thiết bị **ESP32-S3** Giao thức **MQTT · UART · SSE** Backend **Node.js / Fastify** Client **Next.js · Expo**
Edge AI **TinyML int8**

MÔN HỌC	MÃ MÔN	GIẢNG VIÊN
Phát triển ứng dụng IoT	IAD591.22	**Nguyễn Thế Nghĩa, Ph.D**

TRƯỜNG	NGÀY NỘP
FPT School of Business & Technology	11/10/2026

NHÓM 6

Nguyễn Công Cường	25MS33080	**Hồ Khắc Bắc**	25MS33069
Đặng Thành Nhân	25MS33077	**Lê Trọng Quang**	25MS33079



01 Giới thiệu

1.1 Bối cảnh và vấn đề

Một phòng ở hoặc phòng làm việc có thể chuyển sang trạng thái không an toàn rất nhanh: nhiệt độ tăng, không khí ngột ngạt, khí độc tích tụ. Các giải pháp "nhà thông minh" phổ biến thường phụ thuộc hoàn toàn vào cloud. Khi mất Internet, thiết bị trở nên "mù", đúng vào lúc cần phản ứng nhất.



Hình 1.1. Cùng một sự cố, cùng mất kết nối cloud. Phòng chỉ dựa vào cloud thì đứng yên; phòng edge-first tự bật cảnh báo và mở cửa thông gió.

Đồ án xây dựng một hệ thống IoT hoàn chỉnh cho một phòng, từ cảm biến tới ứng dụng, theo hướng **edge-first** [8]: quyết định an toàn được đưa ra tại thiết bị, còn cloud đảm nhận quan sát, lưu trữ và chuyển lệnh của con người.

1.2 Mục tiêu

- Thu thập dữ liệu môi trường theo thời gian thực: nhiệt độ, độ ẩm, ánh sáng, chất lượng không khí, chuyển động.
- Phân loại trạng thái phòng thành 4 mức `NORMAL` `UNCOMFORTABLE` `WARNING` `DANGER` và tự điều khiển cửa sổ, còi, đèn báo ngay tại chỗ.

- Truyền dữ liệu tin cậy qua MQTT và xây dựng *digital twin* phản chiếu đúng trạng thái thiết bị báo lên.
- Cho phép điều khiển hai chiều từ dashboard web, app Android và trợ lý giọng nói dùng LLM.
- Thử nghiệm Edge AI: một mô hình TinyML chạy trên ESP32 thay rule cứng trong điều khiển thường ngày.

1.3 Phạm vi

Hệ thống giám sát **một phòng**. Thiết bị có hai phương án phần cứng: 1 mạch, hoặc 2 mạch (node cảm biến + gateway mạng). Cả hai chạy được trên trình mô phỏng **Wokwi** [11]; phương án 2 mạch đã được lắp và chạy trên **mạch ESP32-S3 thật** (Hình 4.2). Dữ liệu lưu trong RAM hoặc trên Cloud Firestore. Xác thực người dùng và mã hoá TLS nằm ngoài phạm vi của phiên bản này (xem chương 9).

1.4 Đóng góp chính

- Kiến trúc phân lớp đầy đủ, mỗi lớp thay thế được độc lập nhờ một bộ *contract* (schema) dùng chung.
- Edge rule engine có *hysteresis*, phát hiện lỗi cảm biến và cơ chế *manual override* có thời hạn.
- Giao thức link riêng giữa 2 vi điều khiển: frame COBS + CRC-16, tự đồng bộ lại sau nhiễu.
- Mô hình TinyML 552 byte, dữ liệu huấn luyện do LLM gán nhãn theo một chính sách tiện nghi; rule engine vẫn giữ vai trò lớp an toàn cuối.
- Trợ lý giọng nói tiếng Việt và tiếng Anh. Lệnh do AI tạo đi qua đúng các bước kiểm soát như lệnh của người.

1.5 Phân công công việc

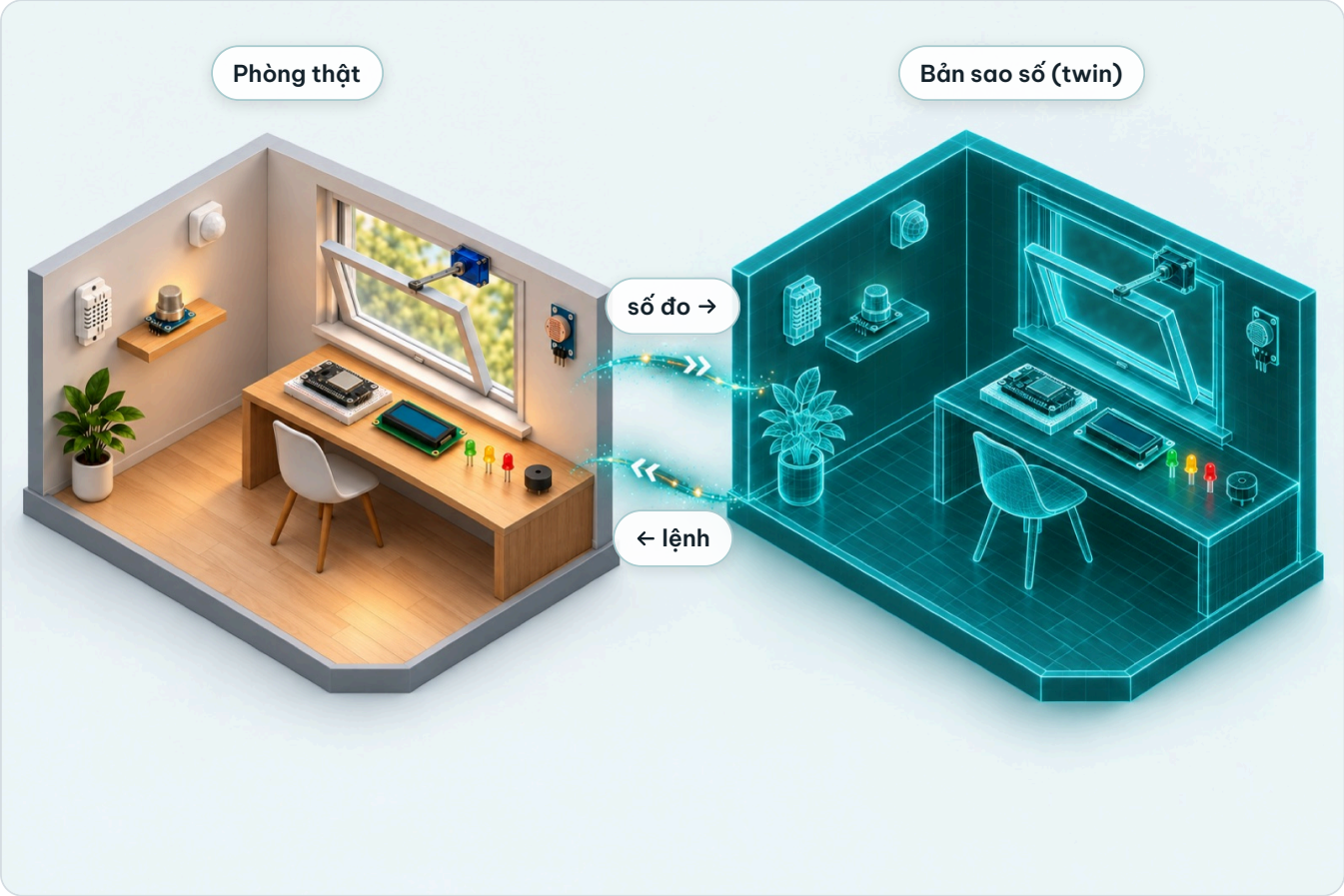
Bảng 1.1. Phân công công việc trong nhóm.

THÀNH VIÊN	MSSV	PHỤ TRÁCH
Lê Trọng Quang	25MS33079	Mô hình TinyML: chính sách gán nhãn, huấn luyện, lượng tử hoá int8, đánh giá (mục 4.4)
Đặng Thành Nhân	25MS33077	Lắp mạch thật: phương án 2 mạch ESP32-S3 trên breadboard, nối cảm biến và cơ cấu chấp hành (Hình 4.2); firmware và giao thức UART giữa node và gateway (mục 4.2–4.3, 5.2)
Nguyễn Công Cường	25MS33080	Dashboard web (mục 7.1)
Hồ Khắc Bắc	25MS33069	Kiến trúc tổng thể, thiết kế mạch trên trình mô phỏng Wokwi, backend, MQTT broker, Cloud Firestore, app Android, trợ lý giọng nói

02 Tổng quan: góc nhìn trực giác

Trước khi vào thuật ngữ, cần nắm hai hình ảnh: bản sao số của căn phòng, và căn phòng như một cơ thể sống.

2.1 Digital twin là gì



Hình 2.1. Twin [7] là một bản sao luôn khớp với phòng thật: góc cửa, đèn, còi, số đo. Dữ liệu đi từ phòng sang twin; lệnh đi ngược lại và chỉ được phản ánh lên twin sau khi phòng thật đã thực hiện.

2.2 Căn phòng như một cơ thể sống

Bảng 2.1. Các thành phần của hệ thống, đối chiếu với một cơ thể sống.

BỘ PHẬN CƠ THỂ	VAI TRÒ	TRONG HỆ THỐNG
Giác quan	Cảm nhận môi trường	DHT22, LDR, cảm biến không khí, PIR, nút nhấn
Phản xạ tuỷ sống	Phản ứng ngay, không chờ não	Edge rules + TinyML trên ESP32
Tay chân	Tác động lại môi trường	Servo cửa sổ, còi, 3 LED, màn hình LCD
Dây thần kinh	Truyền tín hiệu hai chiều	UART, Wi-Fi, MQTT
Não bộ, trí nhớ	Ghi nhận, theo dõi, điều phối	Backend Fastify + Storage
Tấm gương	Phản chiếu cơ thể theo thời gian thực	Digital twin trên web và app
Lời nói	Ra lệnh bằng ngôn ngữ tự nhiên	Trợ lý giọng nói (LLM)

2.3 Một tình huống từ đầu đến cuối

Bốn khung dưới đây kể lại một sự cố, đi qua mọi thành phần của hệ thống.
Hình 2.3 tóm tắt cùng câu chuyện đó dưới dạng sơ đồ.



1 Phòng quá nóng

35 °C, không khí ngột ngạt (AQ 900). Cảm biến đo mỗi 1–2 s.



2 Thiết bị tự phản ứng

ESP32 chuyển DANGER: LED đỏ, còi, mở cửa 90°. Router mất mạng cũng không sao.



3 Dữ liệu lên hệ thống

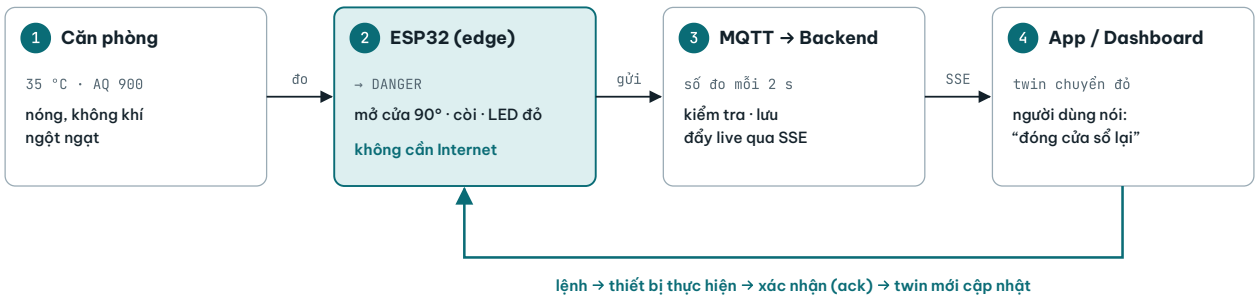
Telemetry qua MQTT tới backend: kiểm tra, lưu, đẩy live qua SSE.



4 Người dùng ra lệnh

Twin trên app chuyển đỏ. Người dùng nói "đóng cửa sổ", lệnh quay về thiết bị.

Hình 2.2. Bốn khoảnh khắc của cùng một sự cố: phòng nóng lên, thiết bị tự xử lý tại chỗ, dữ liệu lên hệ thống, và người dùng ra lệnh từ xa.

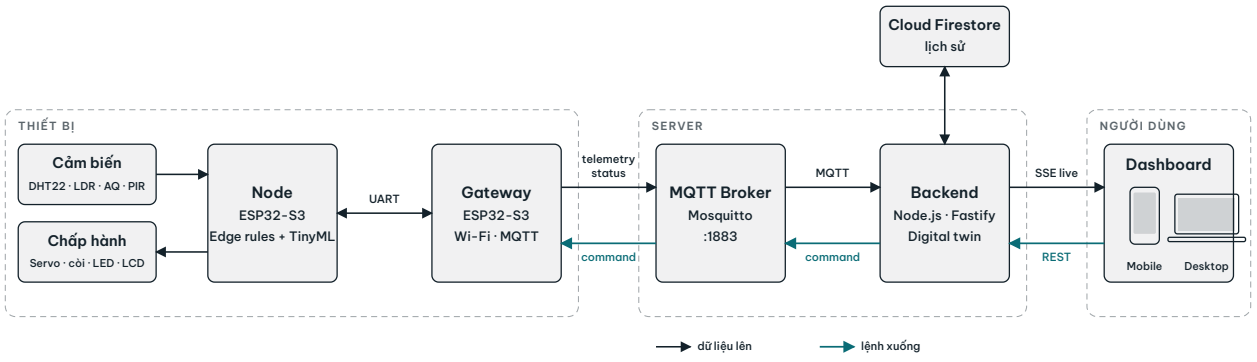


Hình 2.3. Một vòng khép kín. Bước 2 diễn ra hoàn toàn trên thiết bị nên phòng vẫn tự bảo vệ khi mất mạng. Ở chiều ngược lại, giao diện chỉ đổi khi thiết bị đã xác nhận (ack), không đổi theo kiểu "đoán trước".

> **Ý tưởng cốt lõi:** thiết bị tự quyết định (edge-first), cloud quan sát, lưu trữ và chuyển lệnh. Digital twin chỉ phản chiếu sự thật do thiết bị báo lên, không bao giờ tự giả định.

03 Kiến trúc phân lớp

Hệ thống được tổ chức theo mô hình kiến trúc IoT nhiều lớp, có thêm một lớp Edge Computing nằm giữa thiết bị và mạng.



Hình 3.1. Kiến trúc tổng thể theo phương án 2 mạch. Dữ liệu đi từ trái sang phải; lệnh của người dùng (màu nhấn) đi ngược lại qua đúng các chặng đó. Backend là điểm duy nhất nối tới Cloud Firestore và tới các client.



Hình 3.2. Sáu lớp và các thành phần chính của từng lớp. Lớp Edge (viền cam) là điểm khác biệt của hệ thống: quyết định an toàn được đưa ra ở đây, không phải trên server. Dữ liệu đi lên bên trái, lệnh đi xuống bên phải.

3.1 Mỗi lớp làm gì, dùng công nghệ nào

Bảng 3.1. Trách nhiệm và công nghệ của từng lớp.

LỚP	TRÁCH NHIỆM	CÔNG NGHỆ
Perception	Đọc cảm biến, điều khiển servo, còi, đèn	ESP32-S3, C++ (PlatformIO)
Edge	Phân loại trạng thái phòng, quyết định cơ cấu chấp hành	Luật ngưỡng + TinyML (huấn luyện bằng PyTorch)
Network	Vận chuyển bản tin hai chiều	UART, Wi-Fi, MQTT (Mosquitto)
Middleware	Kiểm tra, lưu, theo dõi online, quản lý lệnh, đẩy realtime	Node.js (Fastify), Cloud Firestore
Application	Hiển thị twin, biểu đồ, điều khiển, giọng nói	Next.js (web), Expo / React Native (Android)
Business	Scenes, luật xung đột lệnh, ngưỡng hiển thị	Thư viện TypeScript dùng chung
Xuyên suốt	Một định nghĩa duy nhất cho mọi loại bản tin (single source of truth)	TypeScript

Vị trí mã nguồn của từng lớp được liệt kê ở Phụ lục B.

3.2 Năm nguyên tắc thiết kế

- **Edge-first, local autonomy.** Backend không bao giờ trực tiếp điều khiển actuator theo luật. Các luật phía server chỉ quét offline và quét timeout lệnh.
- **Contract-driven.** Firmware, backend và client dùng cùng một định nghĩa message. Đổi schema chỉ sửa ở một chỗ.
- **Reported state.** Twin = trạng thái thiết bị báo lên gần nhất + số đo mới nhất. Giao diện không "đoán trước" kết quả lệnh (không *optimistic update*).
- **Loose coupling.** Thiết bị và backend nói chuyện qua broker; nơi lưu trữ và đường truyền giữa hai mạch đều thay được mà không sửa phần còn lại.
- **Simulation-first.** Phần nào cũng có bản giả, nên từng người phát triển độc lập mà không cần đủ phần cứng.

04 Lớp thiết bị và Edge Computing

Lớp Perception đo và tác động lên phòng; lớp Edge biến số đo thành quyết định.

4.1 Phần cứng



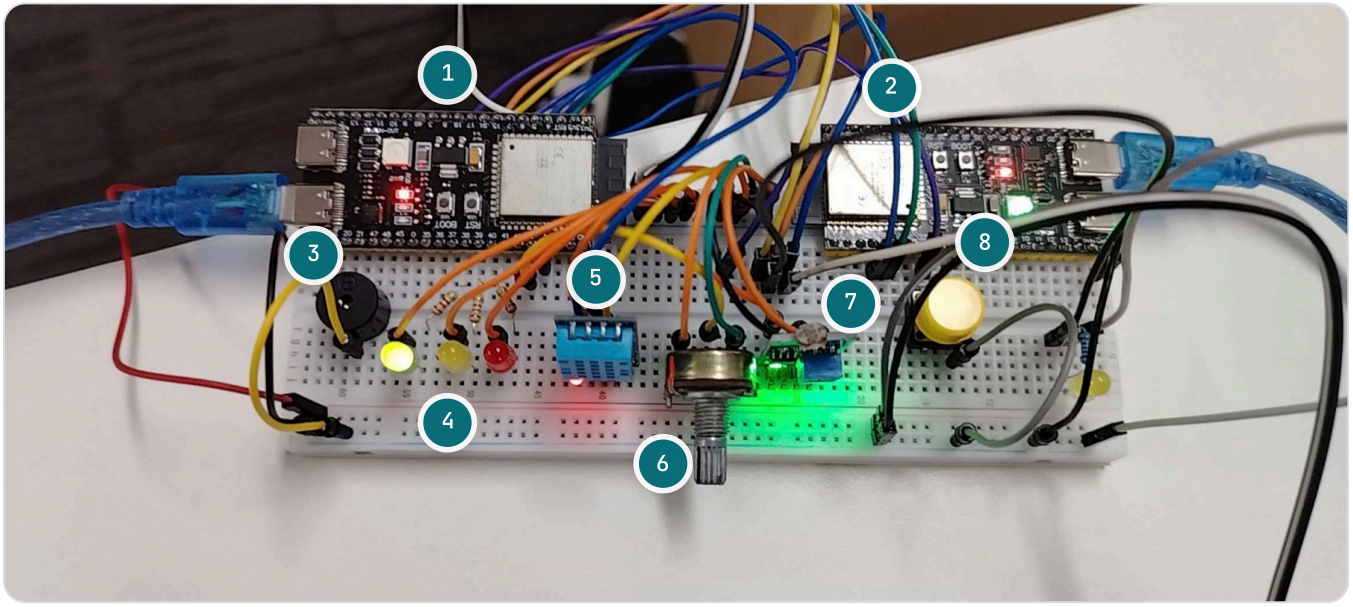
1	DHT22 nhiệt độ, độ ẩm	cảm biến	2	Cảm biến khí	cảm biến
3	PIR chuyển động	cảm biến	4	LDR ánh sáng	cảm biến
5	Servo cửa sổ	chấp hành	6	ESP32-S3	bộ xử lý
7	LCD 1602 I2C	hiển thị	8	LED xanh / vàng / đỏ	hiển thị
9	Buzzer	chấp hành			

Hình 4.1. Vị trí các linh kiện trong phòng. Cảm biến (1–4) là "giác quan", ESP32 (6) là nơi ra quyết định, còn 5, 7, 8, 9 là cơ cấu chấp hành.

Bảng 4.1. Linh kiện và đại lượng đo.

LINH KIỆN	ĐẠI LƯỢNG / VAI TRÒ
DHT22	Nhiệt độ (-40…80 °C), độ ẩm (0…100 %), đọc mỗi 2 s
Module LDR	Ánh sáng, quy đổi ra lux
Biến trở / MQ-135	Chỉ số chất lượng không khí 0…1000
PIR	Có người / không
Nút nhấn	Nhấn ngắn / nhấn giữ 2 s: override tại chỗ
Servo	Cửa sổ 0° (đóng) … 90° (mở hết)
Buzzer	Còi 2 kHz, kêu ngắt quãng 500 ms
LED xanh / vàng / đỏ	Hiển thị trạng thái
LCD 1602 I2C	Số đo, trạng thái, thông điệp

Firmware còn có bản cho bộ kit ESP32 phổ thông (DHT11, OLED, relay thay servo). Sơ đồ chân chi tiết ở Phụ lục B.



1	ESP32-S3 node	bộ xử lý	2	ESP32-S3 gateway	mạng
3	Buzzer	chấp hành	4	LED xanh / vàng / đỏ	hiển thị
5	Cảm biến nhiệt độ, độ ẩm	cảm biến	6	Biến trở (giả lập AQ)	cảm biến
7	Module LDR	cảm biến	8	Nút nhấn	đầu vào

Hình 4.2. Phương án 2 mạch lắp thật trên một breadboard. Node (1) đọc cảm biến và điều khiển còi, LED; gateway (2) lo Wi-Fi và MQTT, LED trạng thái trên bo đang sáng xanh nghĩa là đã kết nối broker. LED xanh lá (4) đang sáng: phòng ở mức NORMAL.

4.2 Tổ chức firmware

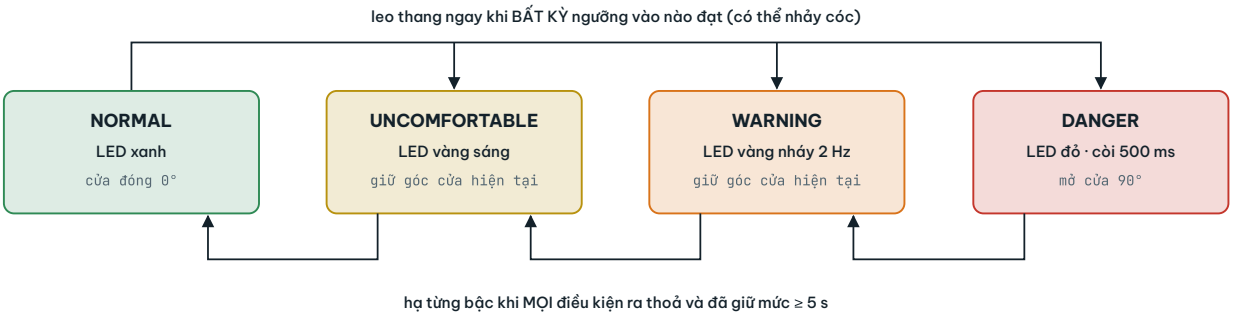
ESP32-S3 có 2 nhân. Firmware chạy trên FreeRTOS [4] và tách việc theo nhân: **nhân 1** chạy vòng lặp chính (cảm biến → rule → actuator), **nhân 0** chạy tác vụ mạng (Wi-Fi, NTP, MQTT). Hai bên trao đổi qua hàng đợi FreeRTOS, nên kết nối mạng chậm hay rớt cũng không làm trễ phản ứng an toàn.

Phần logic (luật edge, giao thức, đường truyền, TinyML) được viết tách khỏi phần cứng, nên kiểm thử tự động được ngay trên máy tính mà không cần mạch thật.

4.3 Edge rule engine

Mỗi giây, rule engine thực hiện ba bước:

1. **Tiền xử lý tín hiệu:** loại giá trị ngoài miền hợp lệ, làm mượt bằng trung bình trượt 3 mẫu. DHT22 hỏng liên tục quá 10 s thì phát cảnh báo lỗi cảm biến.
2. **Cập nhật state machine** 4 mức (Hình 4.3).
3. **Sinh tín hiệu ra** cho LED, còi và góc servo.



Hình 4.3. State machine có hysteresis: lên nhanh, xuống chậm. Ngưỡng ra thấp hơn ngưỡng vào nên trạng thái không nhảy qua lại khi số đo dao động quanh ngưỡng.

Bảng 4.2. Ngưỡng vào / ra của từng mức trạng thái.

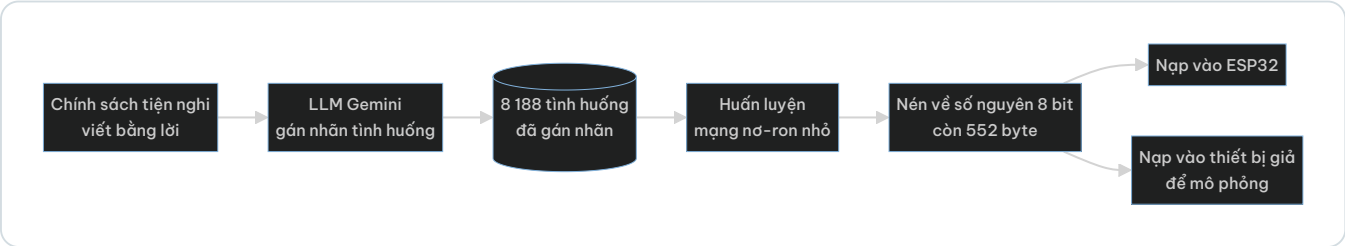
MỨC	NHIỆT ĐỘ VÀO / RA	CHẤT LƯỢNG KK VÀO / RA	ĐỘ ẨM VÀO / RA
UNCOMFORTABLE	≥ 29.0 / < 28.5 °C	≥ 500 / < 470	≥ 75 / < 72 %
WARNING	≥ 31.0 / < 30.0 °C	≥ 700 / < 650	–
DANGER	≥ 34.0 / < 33.0 °C	≥ 900 / < 850	–

Manual override. Khi người dùng mở/đóng cửa hoặc bật/tắt còi (từ app, dashboard, AI hay nút trên mạch), lệnh đó thắng rule engine trong **120 s**,

sau đó quyền điều khiển trở về edge. Override tự huỷ nếu phòng leo thang lên mức cao hơn mức lúc đặt override. LED, sự kiện và trạng thái vẫn cập nhật bình thường trong lúc override.

4.4 TinyML: mô hình điều khiển trên vi điều khiển

Rule engine dùng ngưỡng cứng và không biết ngữ cảnh: phòng trống hay có người, ngày hay đêm, nóng kèm ẩm. Hệ thống bổ sung một mạng nơ-ron nhỏ chạy trực tiếp trên ESP32 (TinyML [5]), đảm nhận điều khiển thường ngày.



Hình 4.4. Pipeline huấn luyện. LLM đóng vai "chuyên gia gán nhãn" theo một chính sách viết sẵn. Cùng một mô hình được nạp vào ESP32 và vào thiết bị giả, và có kiểm thử đảm bảo hai bản cho kết quả giống nhau.

Đầu vào (5 giá trị đã làm mượt): nhiệt độ, độ ẩm, chất lượng không khí, log ánh sáng, có người. **Đầu ra:** trạng thái (4 lớp), góc cửa (một trong 0/20/30/45/60/70/90°), bật còi hay không. Mỗi lần suy luận chỉ khoảng 300 phép nhân-cộng. Trọng số được lượng tử hoá về số nguyên 8 bit [6].

96.7 %	0.965	1.65°	99.1 %
State accuracy (int8)	Macro F1	Sai số góc cửa (MAE)	Buzzer accuracy

Bảng 4.3. Ma trận nhầm lẫn của mô hình int8 trên tập test.

NHÃN \ DỰ ĐOÁN	NORMAL	UNCOMF.	WARNING	DANGER
NORMAL	148	7	0	0
UNCOMFORTABLE	8	389	7	0
WARNING	0	4	368	2
DANGER	0	0	13	286

Tập test gồm 1 232 mẫu (Bảng 4.3). Lỗi chỉ xuất hiện giữa các mức liền kề; không có mẫu NORMAL nào bị đoán thành DANGER hay ngược lại. Mô hình chỉ khớp với rule engine khoảng 86 %. Đây là chủ ý, vì chính sách gán nhãn có thêm ngữ cảnh mà rule không có. Ví dụ: có người thì thông gió sớm từ AQ 380, ban đêm thì giới hạn cửa ở 30°, phòng lạnh dưới 18 °C thì đóng cửa.

4.5 Thứ bậc an toàn

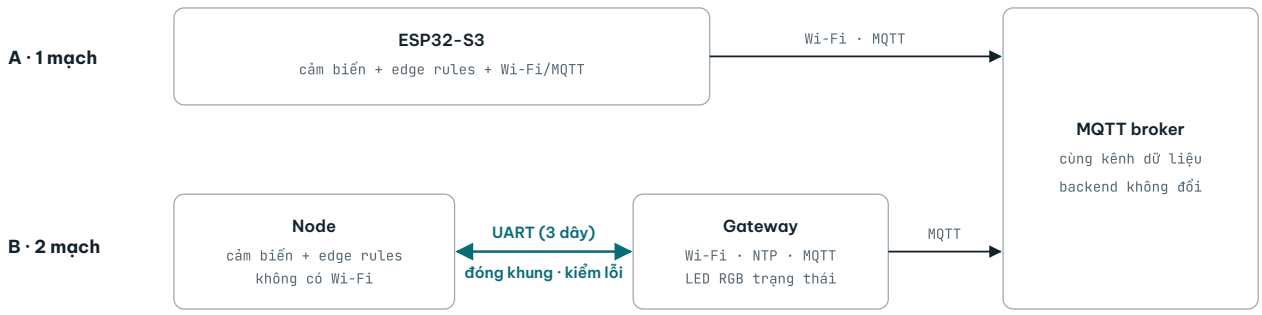
Khi nhiều nguồn cùng muốn điều khiển cửa sổ và còi, firmware phân xử theo thứ tự cố định sau:

1. **Rule engine báo DANGER** → cửa 90°, còi bật, bất kể mô hình nói gì (*safety floor*).
2. **Manual override** trong 120 s, từ người dùng hoặc trợ lý AI.
3. **TinyML** điều khiển thường ngày khi có đủ đầu vào hợp lệ.
4. **Rule engine** khi mô hình bị tắt hoặc cảm biến DHT lỗi.

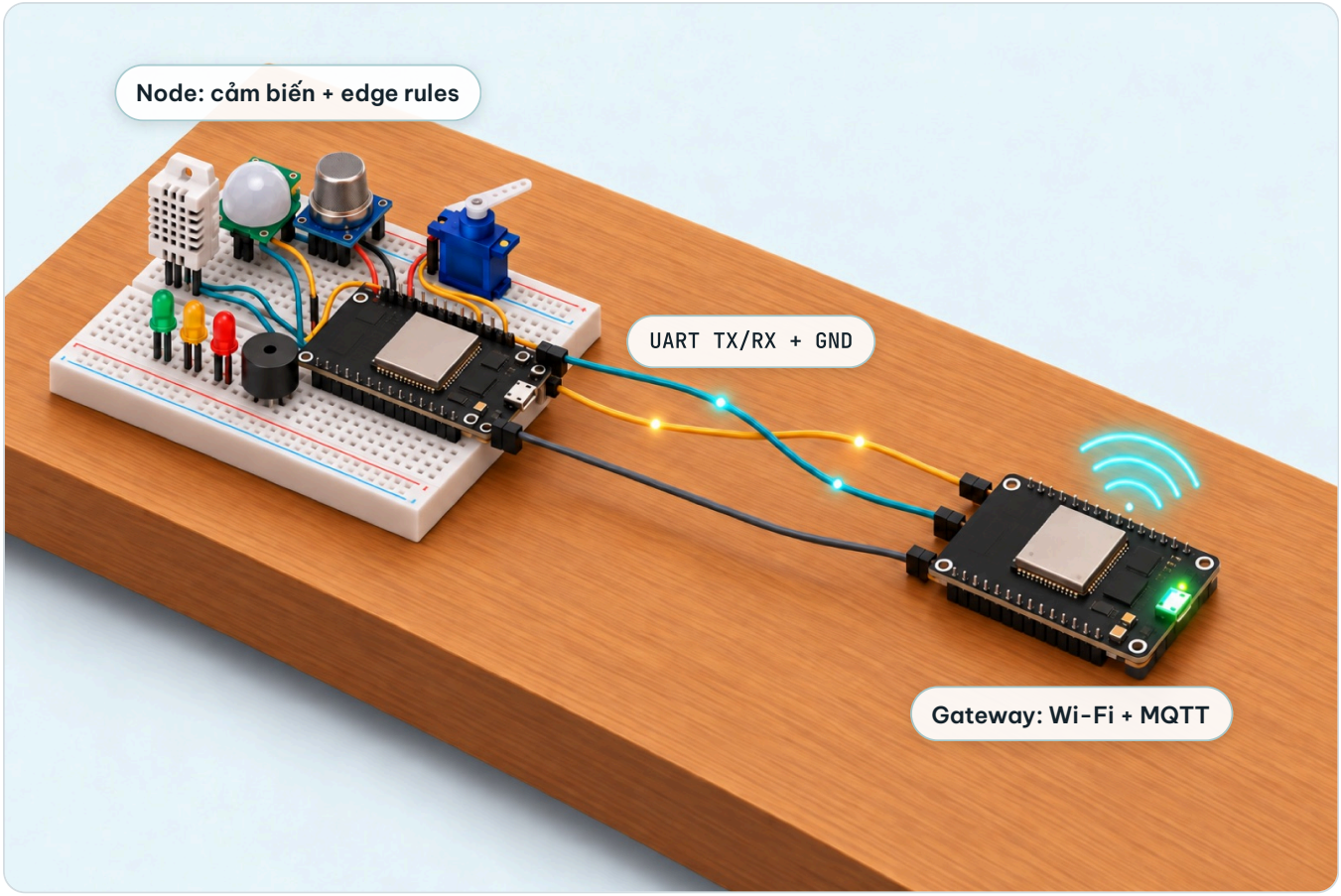
05 Lớp mạng và truyền thông

Có hai chặng truyền: giữa hai vi điều khiển (UART), và từ thiết bị lên hệ thống (MQTT).

5.1 Hai phương án triển khai thiết bị



Hình 5.1. Phương án B thêm đúng một chặng (UART, màu nhấn) để tách phần đo khỏi phần mạng. Gateway chuyển nguyên bản tin lên đúng kênh như phương án A, nên backend và client không phải sửa gì. Node vẫn tự chạy an toàn khi gateway chết.



Hình 5.2. Phương án 2 mạch ngoài đời thực: chỉ cần 3 dây (TX, RX nối chéo và GND chung) giữa node và gateway.

Gateway là một *transparent bridge*: nó không đọc nội dung bản tin, chỉ lo đăng ký node khi node khởi động, theo dõi sự sống hai chiều (mất node quá 15 s thì báo offline thay cho node), đồng bộ giờ cho node (vì node không có Internet) và báo trạng thái qua LED: đỏ = chưa có Wi-Fi, vàng = chưa có MQTT, xanh = sẵn sàng, nháy = mất node.

Hình 4.2 là phương án này khi lắp thật: hai bo ESP32-S3 trên cùng một breadboard, LED trạng thái của gateway đang sáng xanh.

5.2 Giao thức truyền giữa node và gateway



Hình 5.3. Cấu trúc một gói tin trên dây nối hai mạch. Dấu kết thúc không bao giờ xuất hiện ở giữa gói, nên một gói hỏng do nhiễu chỉ làm mất chính gói đó; bên nhận tự bắt nhịp lại ở dấu kết thúc kế tiếp. Cách đóng gói này là COBS [3].

Bên nhận kiểm tra lần lượt độ dài, mã kiểm lỗi, phiên bản và loại bản tin; sai ở bước nào thì bỏ gói đó và không trả lời. Đường truyền này **không xác nhận từng gói**, vì độ tin cậy đã được đảm bảo ở tầng trên: mỗi lệnh có mã riêng và thời hạn, còn mất một gói số đo thì 2 s sau đã có gói mới. Phần truyền được viết tách rời, nên cùng một logic chạy được qua dây UART thật, cáp USB, mạng TCP (khi mô phỏng bằng Wokwi) hoặc qua MQTT.

Bảng 5.1. Các loại bản tin trên đường truyền UART.

LOẠI BẢN TIN	HƯỚNG	GATEWAY LÀM GÌ
Số đo, trạng thái, sự kiện, xác nhận lệnh	node → gateway	chuyển tiếp lên MQTT
Đăng ký	node → gateway	ghi nhận node vừa khởi động
Nhịp tim	node → gateway	xác nhận node còn sống
Lệnh	gateway → node	chuyển lệnh từ MQTT xuống
Đồng bộ giờ	gateway → node	gửi giờ hiện tại
Trạng thái mạng	gateway → node	báo gateway còn sống và đã có MQTT hay chưa

5.3 MQTT

Thiết bị và backend không gọi trực tiếp nhau mà giao tiếp qua broker Mosquitto [2] theo mô hình *publish/subscribe*. Mỗi thiết bị có một nhóm 5 kênh (*topic*) riêng [1].

Bảng 5.2. Năm kênh MQTT của một thiết bị.

KÊNH	HƯỚNG	QOS / RETAIN	NỘI DUNG
Số đo (telemetry)	lên	`0`	Số đo + trạng thái + cơ cấu chấp hành, mỗi 2 s và ngay khi đổi trạng thái
Trạng thái (status)	lên	`0 · retained (LWT: 1 · retained)`	Online + cơ cấu chấp hành, khi đổi và mỗi 30 s; kèm LWT báo offline
Sự kiện (event)	lên	`0`	Khởi động, đổi trạng thái, lỗi cảm biến, đặt override…
Lệnh (command)	xuống	`1`	Lệnh điều khiển, mỗi lệnh có mã riêng
Xác nhận lệnh (ack)	lên	`0`	Đã thực hiện / từ chối / lỗi

- **QoS 0** (at most once) cho telemetry: mất một bản tin không sao vì bản tin mới đến ngay sau đó. **QoS 1** (at least once) cho lệnh và bản tin LWT: phải đến nơi, nếu trùng thì phía nhận tự lọc. Thư viện MQTT trên ESP32 (PubSubClient) chỉ publish được QoS 0, nên status thường ngày đi QoS 0 kèm cờ retained; mất một bản thì 30 s sau có bản mới.
- **Retained message:** broker giữ lại status cuối cùng, nên client vừa kết nối nhận được trạng thái ngay, không phải chờ 30 s.
- **Last Will and Testament (LWT):** thiết bị gửi trước cho broker một bản tin "tôi đã offline". Nếu thiết bị chết đột ngột, broker tự phát bản tin này sau khoảng 20–25 s.

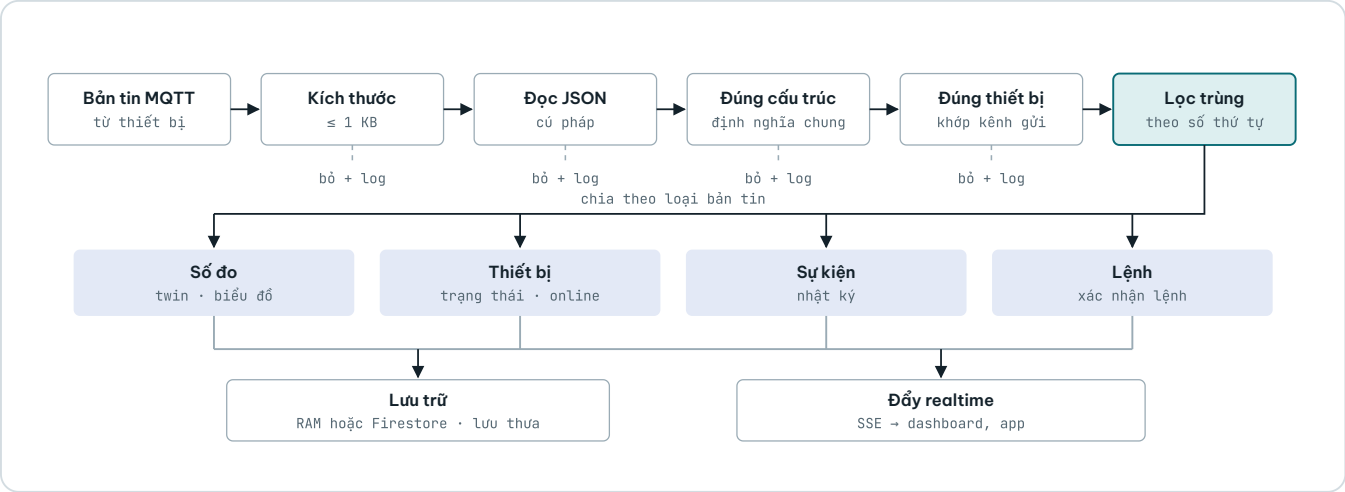
```
y', ... (299 bytes))
1791690564: Received PUBLISH from dev-room-01 (d0, q0, r0, m0, 'srdt/room-01/telemetry', ..
. (299 bytes))
1791690564: Sending PUBLISH to srdt-server-1558fbd3 (d0, q0, r0, m0, 'srdt/room-01/telemetr
y', ... (299 bytes))
1791690564: Sending PUBLISH to srdt-server-d056e1fd (d0, q0, r0, m0, 'srdt/room-01/telemetr
y', ... (299 bytes))
1791690565: Received PINGREQ from srdt-server-1558fbd3
1791690565: Sending PINGRESP to srdt-server-1558fbd3
1791690565: Received PUBLISH from dev-room-01 (d0, q0, r1, m0, 'srdt/room-01/status', ... (
274 bytes))
1791690565: Sending PUBLISH to srdt-server-1558fbd3 (d0, q0, r0, m0, 'srdt/room-01/status',
 ... (274 bytes))
1791690565: Sending PUBLISH to srdt-server-d056e1fd (d0, q0, r0, m0, 'srdt/room-01/status',
 ... (274 bytes))
1791690565: Received PINGREQ from dev-room-01
1791690565: Sending PINGRESP to dev-room-01
1791690566: Received PUBLISH from dev-room-01 (d0, q0, r0, m0, 'srdt/room-01/telemetry', ..
. (299 bytes))
1791690566: Sending PUBLISH to srdt-server-1558fbd3 (d0, q0, r0, m0, 'srdt/room-01/telemetr
y', ... (299 bytes))
1791690566: Sending PUBLISH to srdt-server-d056e1fd (d0, q0, r0, m0, 'srdt/room-01/telemetr
y', ... (299 bytes))
```

Hình 5.4. Log của broker Mosquitto khi mạch thật đang chạy. Thiết bị `dev-room-01` gửi telemetry (299 byte, `q0`) mỗi 2 s và status có cờ retained (`r1`); broker chuyển tiếp từng bản tới backend (`srdt-server-…`). Các cặp PINGREQ / PINGRESP là cơ chế keep-alive mà LWT dựa vào để phát hiện thiết bị chết.

06 Lớp Middleware: backend

Backend là client MQTT duy nhất được phép gửi lệnh, là nơi giữ twin, và là cổng duy nhất để client truy cập dữ liệu.

6.1 Tiếp nhận dữ liệu (ingestion)

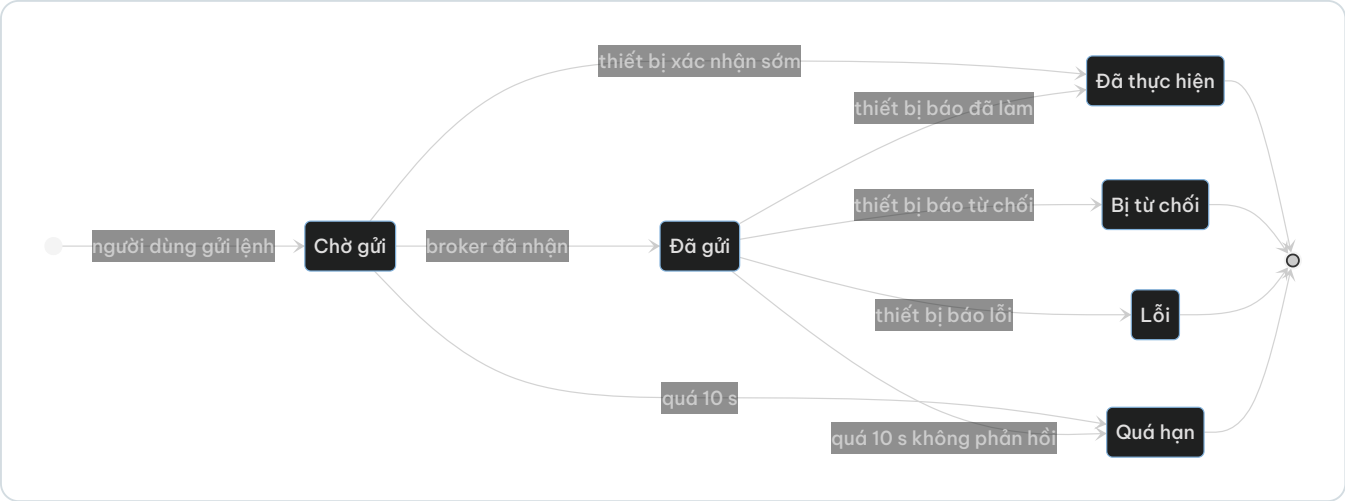


Hình 6.1. Mọi bản tin từ thiết bị đi qua cùng một chuỗi kiểm tra trước khi chạm vào twin. Một bản tin xấu chỉ bị bỏ và ghi log, không làm hỏng service. Mỗi bản tin mang số thứ tự, nên bản tin bị gửi lặp (QoS 1) được nhận ra và không tạo bản ghi trùng.

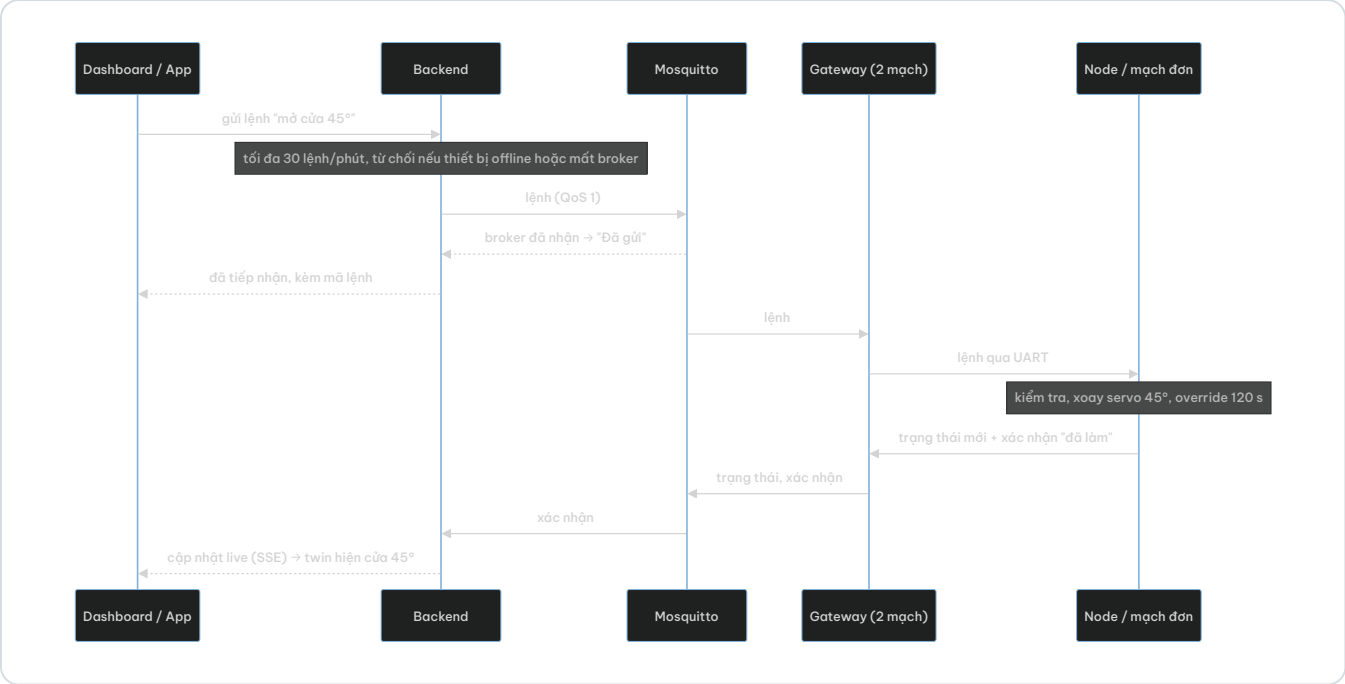
6.2 Device presence

Thiết bị được coi là **online** khi có bản tin thật (bản retained cũ và LWT không tính). Thiết bị chuyển **offline** khi có LWT, hoặc khi không có dữ liệu trong 45 s (backend quét mỗi 10 s). Mỗi lần chuyển trạng thái đều được ghi vào nhật ký sự kiện. Phía client đánh dấu dữ liệu là "cũ" (stale) nếu thiết bị online nhưng đã 15 s không có telemetry.

6.3 Vòng đời của một lệnh



Hình 6.2. Các trạng thái của một lệnh. Mỗi lần chuyển là một phép *compare-and-set* (chỉ chuyển khi trạng thái hiện tại đúng như mong đợi). Nhờ vậy xác nhận của thiết bị đến sớm hơn xác nhận của broker vẫn cho kết quả đúng, và xác nhận đến muộn không ghi đè được trạng thái "Quá hạn".



Hình 6.3. Đường đi của lệnh mở cửa 45°. Với phương án 1 mạch thì bỏ chặng gateway. Trên mạch thật (2 mạch, Wi-Fi trong LAN), thời gian từ "Đã gửi" tới "Đã thực hiện" có trung vị khoảng 0.3 s (mục 8.4).

6.4 Lưu trữ

Phần xử lý không phụ thuộc vào nơi lưu cụ thể (*repository pattern*). Có hai lựa chọn: **RAM** (mặc định, mất khi khởi động lại) và **Cloud Firestore**.

Bảng 6.1. Cách lưu từng loại dữ liệu.

DỮ LIỆU	CÁCH LƯU
Thiết bị	Trạng thái mới nhất của từng thiết bị; thời điểm "thấy lần cuối" ghi tối đa 1 lần / 15 s
Số đo	Mỗi bản ghi có mã cố định theo số thứ tự, nên ghi trùng không tạo bản sao (*idempotent*); chỉ giữ 1 bản ghi mỗi 15 s
Sự kiện	Nhật ký của thiết bị và server, cũng đặt mã theo số thứ tự
Lệnh	Toàn bộ vòng đời của từng lệnh, cập nhật bằng transaction

Tối ưu chi phí. Gói miễn phí giới hạn 20 000 lượt ghi mỗi ngày [10]. Live UI vẫn nhận telemetry mỗi 2 s qua SSE, nhưng bản lưu chỉ giữ 1 bản ghi mỗi 15 s, và trạng thái thiết bị chỉ được ghi khi thực sự thay đổi. Nhờ vậy số lượt ghi đo được giảm từ 13 920 xuống 6 480 lượt/ngày/thiết bị. Bản tổng hợp theo giờ cho trợ lý (mục 7.3) tốn thêm khoảng 170 lượt/ngày, tổng vẫn đủ cho 3 thiết bị trong gói miễn phí.

6.5 Giao tiếp với client

Client chỉ cần bốn thao tác với backend:

1. **Lấy ảnh chụp hiện tại** của phòng: trạng thái thiết bị, số đo 15 phút gần nhất, sự kiện và lệnh gần đây.
2. **Mở một luồng live** để nhận mọi thay đổi ngay khi xảy ra.
3. **Gửi lệnh** và nhận lại mã lệnh để theo dõi.
4. **Gửi câu nói** cho trợ lý và nhận câu trả lời cùng các lệnh đã tạo.

Danh sách API cụ thể ở Phụ lục B.

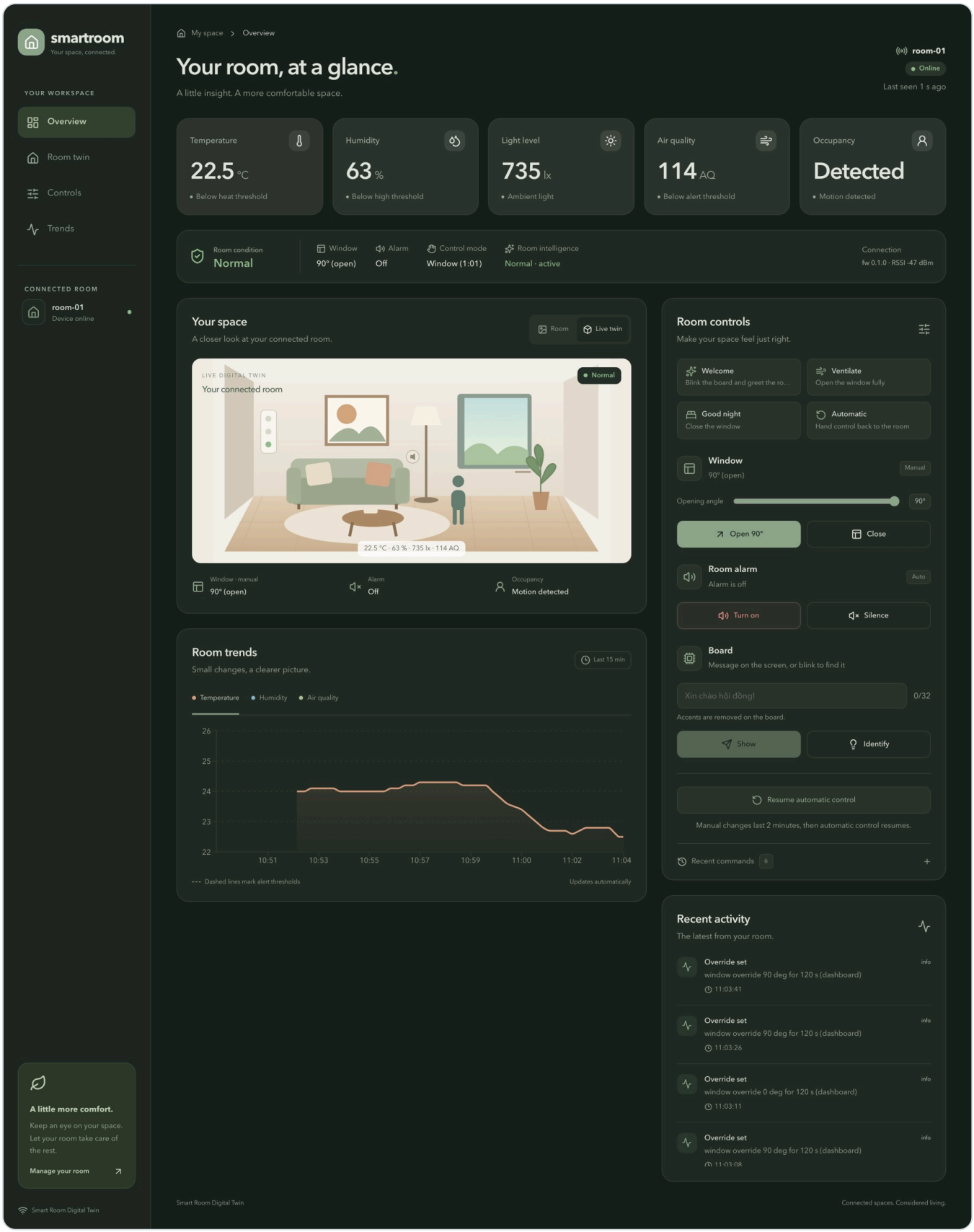
Hệ thống chọn **SSE** [9] thay vì WebSocket vì dữ liệu chỉ cần chảy một chiều từ server xuống; lệnh đi ngược lên bằng REST. SSE chạy trên HTTP thường và trình duyệt tự kết nối lại khi rớt.

07 Lớp ứng dụng và dịch vụ

Hai client ngang hàng, cùng dùng một API và cùng một thư viện logic twin.

7.1 Dashboard web

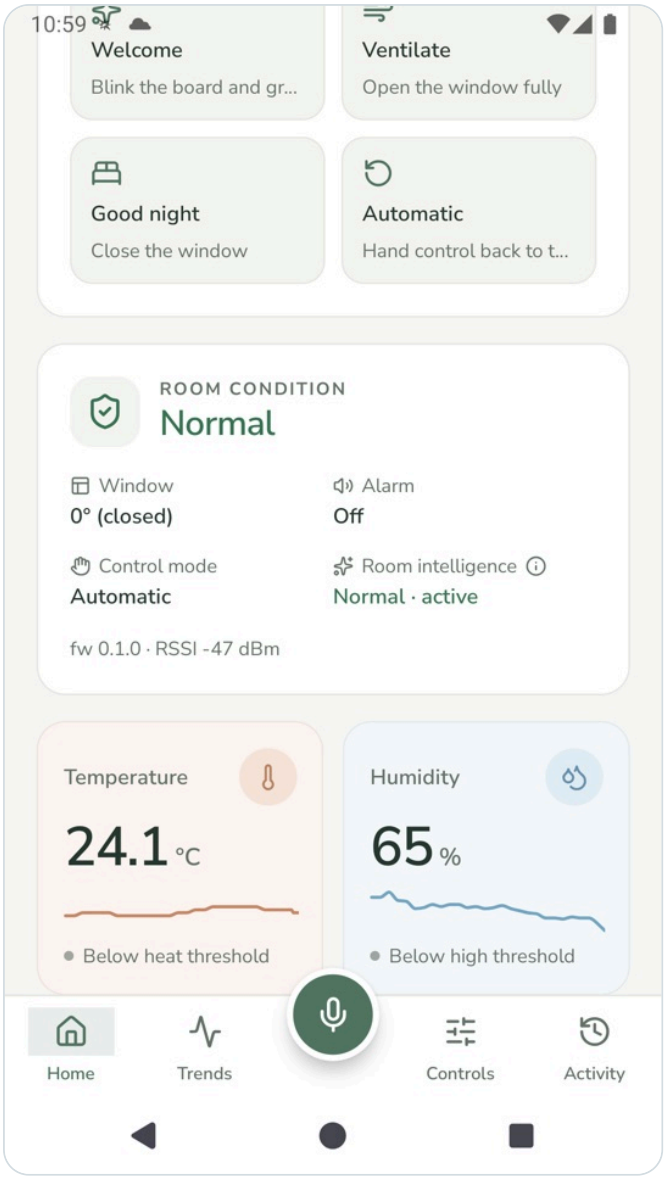
Viết bằng Next.js, xuất dạng static. Dashboard gồm thẻ số đo (tô màu theo ngưỡng), bản vẽ phòng (twin SVG: góc cửa, còi, trạng thái), biểu đồ 15 phút có đường ngưỡng, bảng điều khiển với đếm ngược override, scenes và nhật ký sự kiện. Dữ liệu được tải một lần khi mở trang, sau đó cập nhật liên tục qua luồng live SSE.



Hình 7.1. Dashboard web nối với mạch thật. Cửa sổ vừa được mở tay nên đang ở chế độ override: dải trạng thái hiện "Window (1:27)", tức còn 1 phút 27 giây trước khi trả quyền về edge. Bản vẽ twin phản chiếu cửa đang mở và có người trong phòng, đúng như cảm biến PIR báo lên.

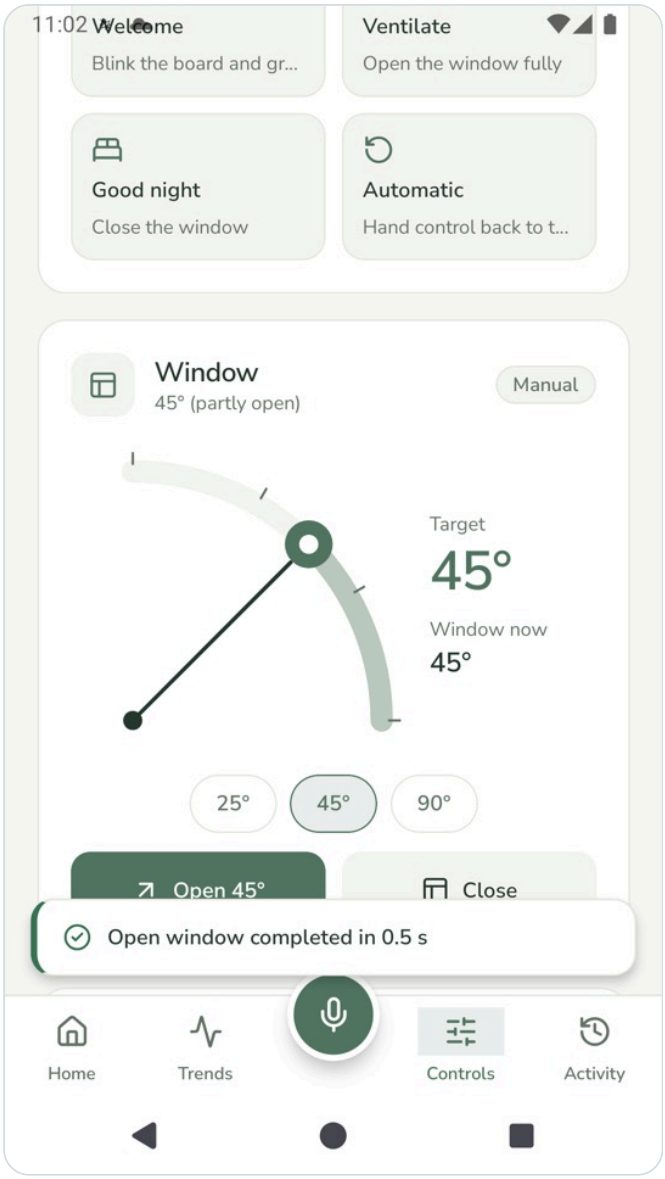
7.2 App Android

Viết bằng Expo / React Native, gồm 4 tab: **Home** (twin + số đo), **Controls** (cửa sổ, còi, scenes, bảng mạch), **Trends** (biểu đồ) và **Activity** (sự kiện), cộng thêm màn Settings. App nhận cập nhật live giống dashboard và lưu cài đặt ngay trên máy.



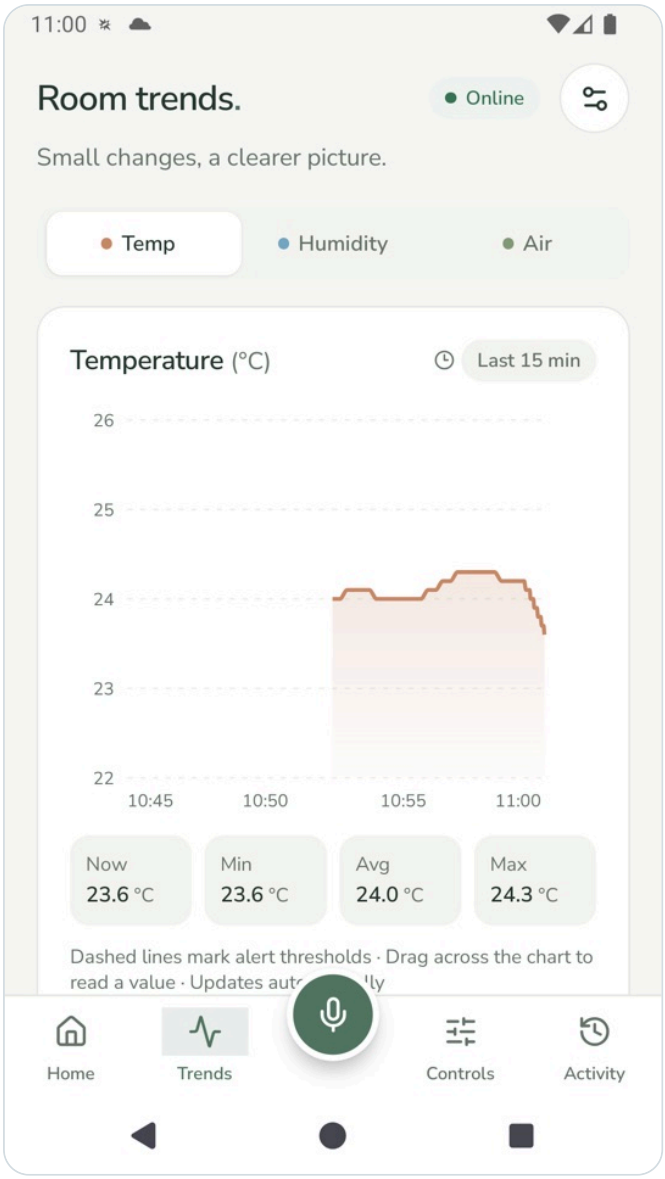
Home

trạng thái phòng và số đo



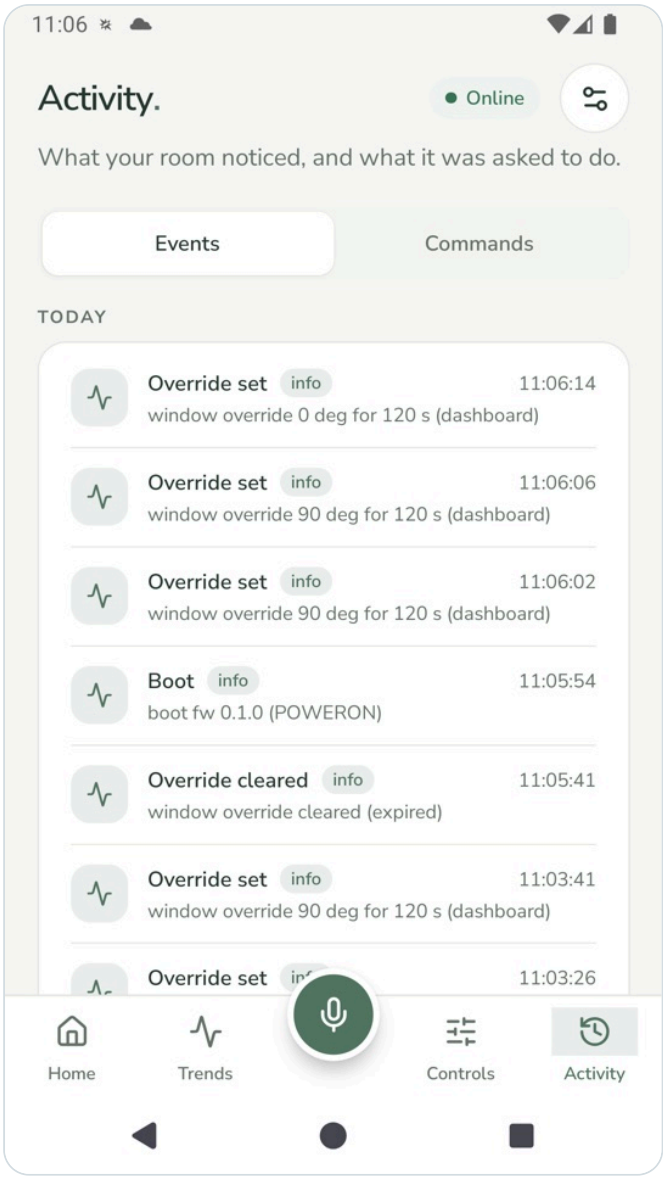
Controls

lệnh mở cửa 45° hoàn tất sau 0.5 s



Trends

biểu đồ 15 phút và thống kê



Activity

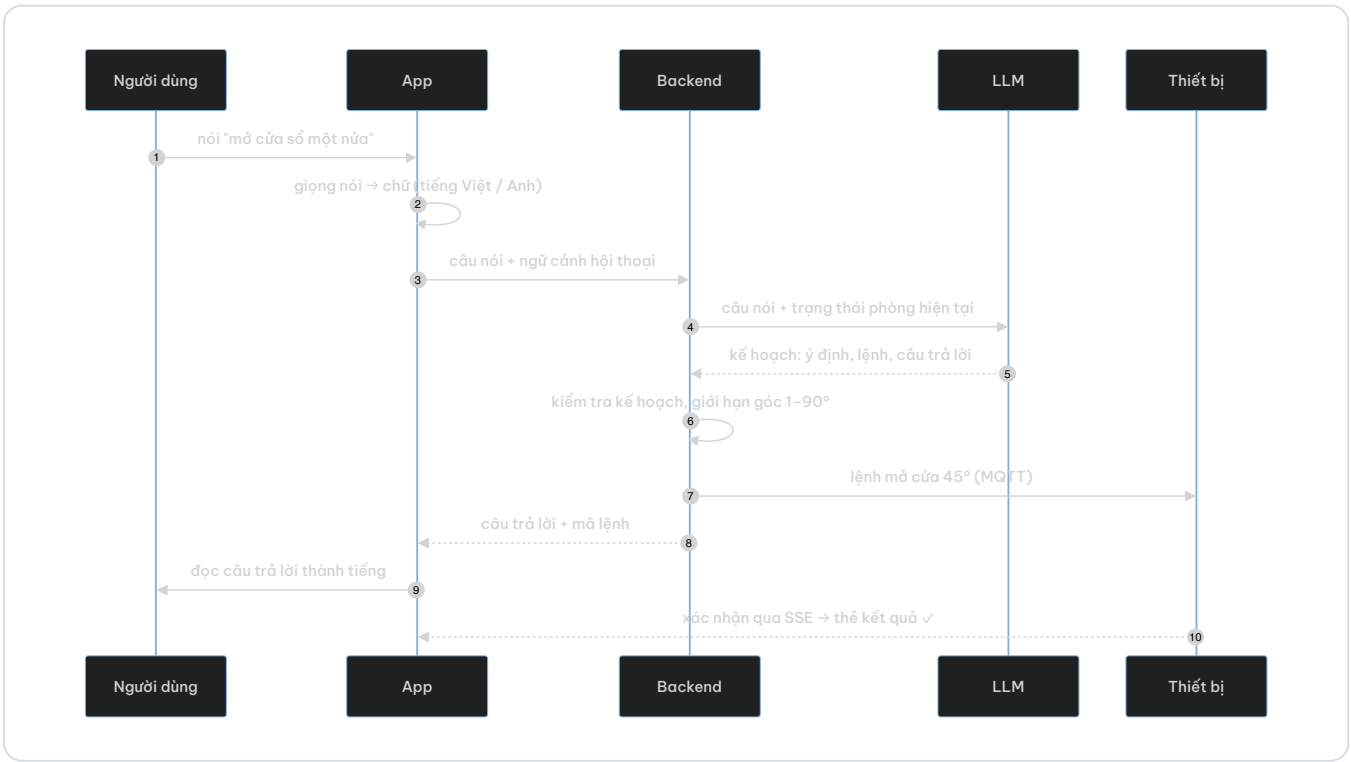
nhật ký sự kiện của thiết bị

Hình 7.2. Bốn màn chính của app, chụp trên emulator Android khi đang nối với mạch thật. Ở màn Controls, thời gian 0.5 s được tính từ lúc bấm tới lúc nhận xác nhận (ack) của thiết bị.

7.3 Trợ lý giọng nói



Hình 7.3. Câu nói trở thành lệnh thật: điện thoại nhận giọng nói, LLM hiểu ý, cửa sổ mở, và app nhận xác nhận từ thiết bị.



Hình 7.4. Một lượt hội thoại ra lệnh. LLM chỉ đề xuất kế hoạch; backend tự kiểm tra và tạo lệnh. Vì vậy lệnh của AI chịu đúng giới hạn tần suất, xác nhận, thời hạn và lịch sử như lệnh tay.

Hệ thống đặt LLM ở server vì ba lý do: khoá API không nằm trong APK; server có sẵn trạng thái twin mới nhất để đưa vào prompt; và mọi lệnh đều đi qua cùng một cổng kiểm soát. Mỗi lần gọi LLM trả về một kế hoạch có cấu trúc cố định (ý định, lệnh, câu trả lời), thay vì để AI tự gọi công cụ qua lại nhiều bước. Câu ra lệnh hay hỏi về hiện tại chỉ cần **một** lần gọi. Cách này nhanh hơn và dễ kiểm thử hơn. Có thể đổi giữa Gemini, Claude và một bộ luật cố định (chạy được khi không có mạng).

Hỏi về quá khứ. Trợ lý trả lời được câu hỏi về **tối đa 30 ngày** gần nhất, ví dụ "tuần này phòng có lúc nào nguy hiểm không?". Khi đó LLM được gọi hai lần. Lần đầu, nó cho biết cần bao nhiêu ngày dữ liệu. Backend lấy số liệu của khoảng đó, nén thành vài chục dòng (min / trung bình / max theo ngày hoặc theo giờ, tỉ lệ thời gian ở từng mức, các sự kiện đáng chú ý, các lần mất kết nối) rồi gọi lại để LLM trả lời có ngày giờ và con số cụ thể. Để không phải đọc hàng trăm nghìn số đo gốc, backend gộp mọi bản tin telemetry thành **bản tổng hợp theo giờ** (khoảng 720 bản ghi cho 30 ngày).

Giới hạn phạm vi. Prompt mô tả đúng các thiết bị có thật trong phòng (cửa sổ, còi, màn hình). Câu hỏi ngoài phạm vi, chẳng hạn bật điều hoà hay hỏi chuyện không liên quan, sẽ bị từ chối lịch sự, kèm gợi ý những việc trợ lý làm được.

7.4 Dịch vụ: scenes

Bảng 7.1. Các scene và chuỗi lệnh tương ứng.

SCENE	CÁC LỆNH GỬI LẦN LƯỢT
Welcome	Mạch nháy đèn và bíp chào → LCD hiện "Welcome!"
Ventilate	Mở cửa 90° → LCD hiện "Airing the room"
Good night	Đóng cửa → LCD hiện "Good night"
Automatic	Trả quyền điều khiển cho chế độ tự động → LCD hiện "Auto mode"

Scene chỉ tồn tại ở phía client. Thiết bị không biết scene là gì, chỉ nhận các lệnh thường, nên không phải sửa firmware khi thêm scene mới.

08 Độ tin cậy, kiểm thử và thực nghiệm

8.1 Cơ chế tin cậy

Bảng 8.1. Cơ chế tin cậy theo từng loại sự cố.

VẤN ĐỀ	CƠ CHẾ	Ở ĐÂU
Mất Internet	Quyết định an toàn chạy tại edge	Firmware
Số đo nhiễu, dao động quanh ngưỡng	Trung bình trượt + hysteresis + giữ mức 5 s	Edge rules
Cảm biến hỏng	Báo lỗi cảm biến; mô hình nhường cho rule	Edge
Nhiễu trên dây UART	CRC-16 + COBS tự đồng bộ lại	Link
Thiết bị chết đột ngột	LWT + quét 45 s + heartbeat của gateway	MQTT, backend
Bản tin lặp do QoS 1	Lọc trùng theo số thứ tự; ghi trùng không tạo bản sao	Backend, Firestore
Lệnh không đến nơi	QoS 1 + ack + timeout 10 s	Backend
Các xác nhận đến lệch thứ tự	Compare-and-set khi chuyển trạng thái	Backend
Spam lệnh (kể cả từ AI)	Rate limit 30 lệnh/phút/thiết bị	Backend
Dữ liệu sai định dạng	Định nghĩa bản tin dùng chung, giới hạn 1 KB	Toàn hệ thống

8.2 Mô phỏng: thay phần nào bằng đồ giả

Mỗi thành phần đều có bản giả: **fake-device** thay toàn bộ thiết bị và có sẵn các kịch bản nóng, khói, ẩm, ban đêm...; **fake-node** / **fake-gateway** thay từng mạch; **splice** nối hai mạch Wokwi và giả lập rút dây. Backend và client chạy giống hệt ở mọi tổ hợp, vì chúng chỉ nhìn thấy bản tin MQTT.

8.3 Kiểm thử tự động

Bảng 8.2. Các bộ kiểm thử tự động.

BỘ TEST	SỐ TEST	NỘI DUNG
Firmware (chạy trên PC)	116	Luật edge, giao thức, đóng gói gói tin, gateway/node, TinyML
Định nghĩa bản tin	32	Cấu trúc của mọi loại bản tin
Backend	84 + 12	Tiếp nhận dữ liệu, vòng đời lệnh, lưu trữ, tổng hợp lịch sử, TinyML khớp giữa hai bản, trợ lý. 12 test chỉ chạy khi có Firestore emulator
Logic twin dùng chung	19	Ngưỡng hiển thị, scenes, luật xung đột lệnh
Mô phỏng đường truyền	23	Đóng gói, đường truyền, gateway, nối hai mạch

Tổng cộng **274 test**, tất cả đều pass ở lần chạy gần nhất (tháng 10/2026). Phần lưu trữ Firestore được kiểm thử lại toàn bộ trên Firestore emulator. Ngoài unit test, trợ lý có một bộ câu hỏi mẫu (eval) để kiểm tra ý định và lệnh mà LLM trả về.

8.4 Kết quả thực nghiệm

- Luồng end-to-end thiết bị → dashboard → lệnh ngược lại chạy được với 1 mạch và 2 mạch trên Wokwi, và với 2 mạch ESP32-S3 thật (Hình 4.2).
- Độ trễ lệnh trên mạch thật, đo trên 20 lệnh liên tiếp (mở/đóng cửa, bật/tắt còi), tất cả đều được thực hiện:

  0.29 s
  Trung vị: broker nhận → thiết bị xác nhận

  0.24–0.44 s
  Khoảng dao động của chặng trên

  0.40 s
  Trung vị: người bấm → thiết bị xác nhận

  20 / 20
  Lệnh ở trạng thái "Đã thực hiện"

  Với thiết bị giả chạy trên máy local, chặng broker → xác nhận chỉ khoảng 3 ms; phần chênh lệch trên mạch thật là thời gian đi qua Wi-Fi, gateway và chặng UART tới node.
- Rút dây link 20 s: sau 15 s dashboard báo offline, nối lại thì online.

- Tắt broker: thiết bị kết nối lại với backoff 1–16 s, mỗi lần mất kết nối chỉ kết nối lại đúng một lần.
- TinyML: kịch bản nóng ẩm → UNCOMFORTABLE, cửa 45° (rule engine vẫn để 0°); AQ 627 có người → cửa 60°.
- Firestore: dữ liệu và biểu đồ vẫn còn sau khi khởi động lại backend; số lượt ghi nằm trong gói miễn phí.

09 Hạn chế và hướng phát triển

Bảng 9.1. Hạn chế hiện tại và hướng phát triển.

HẠNG MỤC	HIỆN TRẠNG	HƯỚNG TIẾP THEO
Xác thực người dùng	`chưa có`	Firebase Auth cho dashboard, app và API
Bảo mật broker	`chưa TLS`, chỉ chạy trong LAN	TLS + user/password (HiveMQ Cloud, EMQX hoặc Mosquitto trên VM)
2 mạch thật	`đã chạy` trên breadboard, nguồn USB	Đóng hộp, chạy liên tục nhiều ngày để đo độ ổn định
Triển khai cloud	`Firestore xong`; Hosting, Cloud Run mới là mock	Deploy backend lên Cloud Run, dashboard lên Firebase Hosting
Nhiều node	`1 node / gateway`	Gateway đa node, hoặc ESP-NOW thay UART
Buffer khi mất mạng	`bỏ telemetry` khi offline	Lưu tạm trên flash, gửi bù khi có mạng
Twin	`SVG 2D`	Twin 3D (three.js), giữ nguyên dữ liệu đầu vào
Cảm biến khí	`biến trở giả lập / MQ-135 tương đối`	Cảm biến hiệu chuẩn (CO_2 NDIR, PM2.5)
Nhãn huấn luyện TinyML	`do LLM gán`, chưa có người kiểm tra lại	Lấy mẫu một phần nhãn cho người duyệt; thu thêm dữ liệu từ phòng thật
Quy mô thực nghiệm	`20 lệnh`, một phòng, phiên chạy ngắn	Đo độ trễ trên nhiều lệnh hơn, có tải mạng, trong nhiều ngày

Hai dòng cuối là giới hạn của phần đánh giá chứ không phải của hệ thống. Độ chính xác 96.7 % của TinyML là mức khớp với chính sách gán nhãn, chưa phải mức "đúng" theo đánh giá của người ở trong phòng. Độ trễ 0.3 s đo trong LAN với một thiết bị, nên chưa nói lên hành vi khi có nhiều thiết bị hoặc mạng chậm.

10 Kết luận

Đồ án đã xây dựng một hệ thống IoT hoàn chỉnh cho một phòng, chạy được trên trình mô phỏng và trên mạch ESP32-S3 thật. Bảng 10.1 đối chiếu với các mục tiêu ở mục 1.2.

Bảng 10.1. Đối chiếu mục tiêu với kết quả.

MỤC TIÊU	KẾT QUẢ
Thu thập dữ liệu môi trường theo thời gian thực	`đạt` 5 đại lượng, gửi mỗi 2 s, hiển thị live trên web và app
Phân loại 4 mức và tự điều khiển tại chỗ	`đạt` state machine có hysteresis chạy trên ESP32, vẫn hoạt động khi mất mạng
Truyền tin cậy qua MQTT, digital twin đúng trạng thái báo lên	`đạt` QoS, LWT, lọc trùng, twin chỉ đổi khi có ack; 20 / 20 lệnh thực hiện trên mạch thật
Điều khiển hai chiều từ web, app và giọng nói	`đạt` trung vị 0.4 s từ lúc bấm tới khi thiết bị xác nhận; trợ lý trả lời được cả câu hỏi về 30 ngày gần nhất
Thử nghiệm Edge AI trên ESP32	`đạt` mô hình 552 byte, độ chính xác 96.7 %, rule engine vẫn là lớp an toàn cuối

Điểm nhóm rút ra rõ nhất là cách phân vai giữa thiết bị và cloud. Khi quyết định an toàn nằm ở edge, phần cloud chỉ còn quan sát, lưu trữ và chuyển lệnh, nên trở nên đơn giản hơn và hỏng phần cloud không làm phòng mất an toàn. Bộ định nghĩa bản tin dùng chung cho phép firmware, backend và hai client phát triển song song mà vẫn khớp nhau; nhờ có thiết bị giả, phần lớn hệ thống được kiểm thử trước khi có mạch thật.

Để đưa hệ thống ra dùng thật, việc cần làm trước tiên là xác thực người dùng và mã hoá kết nối tới broker (chương 9), sau đó là chạy liên tục nhiều ngày trên phần cứng đóng hộp.

TL Tài liệu tham khảo

1. OASIS. *MQTT Version 3.1.1*. OASIS Standard, 2014. docs.oasis-open.org/mqtt/mqtt/v3.1.1
2. Eclipse Foundation. *Eclipse Mosquitto: an open source MQTT broker*. mosquitto.org
3. S. Cheshire, M. Baker. "Consistent Overhead Byte Stuffing". *IEEE/ACM Transactions on Networking*, 7(2), 1999, pp. 159–172.
4. Espressif Systems. *ESP32-S3 Technical Reference Manual* và *ESP-IDF Programming Guide* (FreeRTOS, ADC). docs.espressif.com

5. P. Warden, D. Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly, 2019.
6. B. Jacob et al. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference". *CVPR*, 2018.
7. M. Grieves, J. Vickers. "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems". In *Transdisciplinary Perspectives on Complex Systems*, Springer, 2017.
8. W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu. "Edge Computing: Vision and Challenges". *IEEE Internet of Things Journal*, 3(5), 2016.
9. WHATWG. *HTML Living Standard: Server-sent events*. html.spec.whatwg.org
10. Google. *Cloud Firestore documentation: quotas and pricing*. firebase.google.com/docs/firestore
11. Wokwi. *ESP32 simulator documentation*. docs.wokwi.com

A Thuật ngữ

Bảng A.1. Thuật ngữ.

THUẬT NGỮ	NGHĨA TRONG HỆ THỐNG NÀY
Digital twin	Bản sao số của phòng: trạng thái thiết bị báo lên gần nhất + telemetry mới nhất
Telemetry	Bản tin số đo định kỳ từ thiết bị
Edge computing	Xử lý và ra quyết định ngay tại thiết bị, gần nguồn dữ liệu
Hysteresis	Ngưỡng vào khác ngưỡng ra để trạng thái không dao động
Actuator	Cơ cấu chấp hành: servo, còi, LED, LCD
Publish / subscribe	Mô hình giao tiếp qua broker; bên gửi và bên nhận không biết nhau
QoS 0 / 1	Mức đảm bảo giao bản tin của MQTT: tối đa một lần / ít nhất một lần
Retained message	Bản tin cuối cùng của topic được broker giữ lại cho client mới
LWT	Bản tin "di chúc" broker tự phát khi client mất kết nối bất thường
Presence	Trạng thái online / offline của thiết bị
Ack	Xác nhận thiết bị đã thực thi, từ chối hay lỗi khi nhận lệnh
Idempotent	Thực hiện lặp nhiều lần cho cùng kết quả như một lần
Compare-and-set	Chỉ chuyển trạng thái khi trạng thái hiện tại đúng như mong đợi
COBS	Cách mã hoá để byte 0x00 chỉ xuất hiện ở cuối frame
CRC-16	Mã kiểm tra lỗi 16 bit cho frame
SSE	Server-Sent Events: server đẩy sự kiện một chiều qua HTTP
TinyML	Mô hình học máy đủ nhỏ để chạy trên vi điều khiển
Quantization (int8)	Lượng tử hoá: lưu trọng số bằng số nguyên 8 bit để giảm kích thước
Manual override	Lệnh của người tạm thời thắng điều khiển tự động
Safety floor	Lớp an toàn cuối: rule DANGER luôn thắng mô hình

B Chi tiết kỹ thuật

Dành cho người muốn đối chiếu với mã nguồn. Không cần đọc phần này để theo dõi báo cáo.

B.1 Mã nguồn theo lớp

Bảng B.1. Mã nguồn theo lớp.

LỚP	THƯ MỤC	CÔNG NGHỆ
Perception	`device/src`	C++, PlatformIO, Arduino core 2.x
Edge	`device/lib/edge_rules`, `device/lib/edge_ai`, `ml/`	C++ thuần, PyTorch (huấn luyện)
Network	`device/lib/link`, `broker/`	UART, Wi-Fi, MQTT 3.1.1, Mosquitto
Middleware	`server/`, `cloud/`	Node.js 24, Fastify, mqtt.js, Firestore
Application	`dashboard/`, `mobile/`	Next.js 16, Expo SDK 55, React Native
Business	`packages/twin-core`	TypeScript
Xuyên suốt	`packages/contracts`	TypeScript + Zod

B.2 Sơ đồ chân ESP32-S3

Bảng B.2. Sơ đồ chân ESP32-S3.

LINH KIỆN	CHÂN
DHT22	GPIO15
Module LDR	GPIO1 (ADC1)
Biến trở / MQ-135	GPIO2 (ADC1)
PIR	GPIO16
Nút nhấn	GPIO17
Servo	GPIO18
Buzzer (PWM)	GPIO21
LED xanh / vàng / đỏ	GPIO38 / 39 / 40
LCD 1602 I2C	SDA 8 · SCL 9

Chân analog dùng ADC1 vì ADC2 của ESP32 không dùng được khi Wi-Fi đang bật [4].

B.3 Topic MQTT

Bảng B.3. Topic MQTT.

TOPIC	QOS / RETAIN	GHI CHÚ
`srdt/{deviceId}/telemetry`	0	mỗi 2 s
`srdt/{deviceId}/status`	0 · retained (LWT: 1)	LWT `{"online":false}`
`srdt/{deviceId}/event`	0	`BOOT`, `STATE_CHANGED`, `SENSOR_FAULT`, `OVERRIDE_SET`…
`srdt/{deviceId}/command`	1	`OPEN_WINDOW` (1–90), `CLOSE_WINDOW`, `BUZZER_ON`, `BUZZER_OFF`, `CLEAR_OVERRIDE`, `IDENTIFY`, `SHOW_MESSAGE`
`srdt/{deviceId}/command/ack`	0	`executed` / `rejected` / `failed`

Mỗi bản tin mang `bootId` và `seq`; backend khử trùng theo cặp `(bootId, seq)`, và Firestore dùng `{bootId}-{seq}` làm document id.

B.4 Frame của link UART (v0.2)

Frame thô:
`version (1 B, 0x01) | type (1 B) | seq (1 B) | payload (0–1024 B) | CRC-16/CCITT (2 B, lo trước)`.
Trên dây, frame được mã hoá COBS và kết thúc bằng byte `0x00`; tốc độ
115200 baud.

Bảng B.4. Mã loại frame của link UART.

TYPE	TÊN	HƯỚNG
0x01–0x04	`TELEMETRY`, `STATUS`, `EVENT`, `ACK`	node → gw
0x05	`HELLO`	node → gw
0x06	`HEARTBEAT`	node → gw
0x81	`COMMAND`	gw → node
0x82	`TIME`	gw → node
0x83	`LINK_STATE`	gw → node

Ví dụ HEARTBEAT seq 7: raw `01 06 07 ED 21` → wire
`06 01 06 07 ED 21 00`.

B.5 REST API và SSE

Bảng B.5. REST API và SSE.

METHOD	PATH	MỤC ĐÍCH
GET	`/api/devices/:code`	Snapshot: thiết bị + telemetry 15 phút + 20 event + 20 lệnh
GET	`/api/devices/:code/stream`	SSE: `snapshot` đầu tiên, sau đó `device` / `telemetry` / `event` / `command`
POST	`/api/devices/:code/commands`	Gửi lệnh → 202 / 409 (offline) / 429 (rate limit) / 503 (mất broker)
POST	`/api/devices/:code/assistant`	Câu nói → ý định, câu trả lời, các lệnh đã tạo
GET	`/health`	Trạng thái MQTT, storage, số client SSE

B.6 Cloud Firestore

Bảng B.6. Cấu trúc Cloud Firestore.

COLLECTION	DOCUMENT ID
`devices/{code}`	mã thiết bị
`devices/{code}/telemetry/{id}`	`{bootId}-{seq}`
`devices/{code}/events/{id}`	`{bootId}-{seq}`
`devices/{code}/rollups/{hour}`	giờ UTC, ví dụ `2026-10-11T08`; min / max / tổng mỗi cảm biến và số mẫu theo mức trong giờ đó
`commands/{id}`	uuid

B.7 Lệnh chạy demo

Bảng B.7. Lệnh chạy demo.

VIỆC	LỆNH
Unit test firmware trên PC	`pio test -e native`
Mô phỏng 1 mạch	Wokwi, môi trường `esp32-s3`
Mô phỏng 2 mạch	Wokwi `node-wokwi` + `gateway-wokwi`, rồi `pnpm --filter @srdt/link-sim splice` (giả lập rút dây: `cut 20`)
Theo dõi MQTT	`mosquitto_sub -h 127.0.0.1 -t 'srdt/#' -v`
Thiết bị giả	preset `calm`, `hot`, `smoke`, `humid`, `night`...
Chọn LLM cho trợ lý	`ASSISTANT_PROVIDER=gemini | anthropic | rules`